Ejercicio práctico: control de versiones en Amazon S3

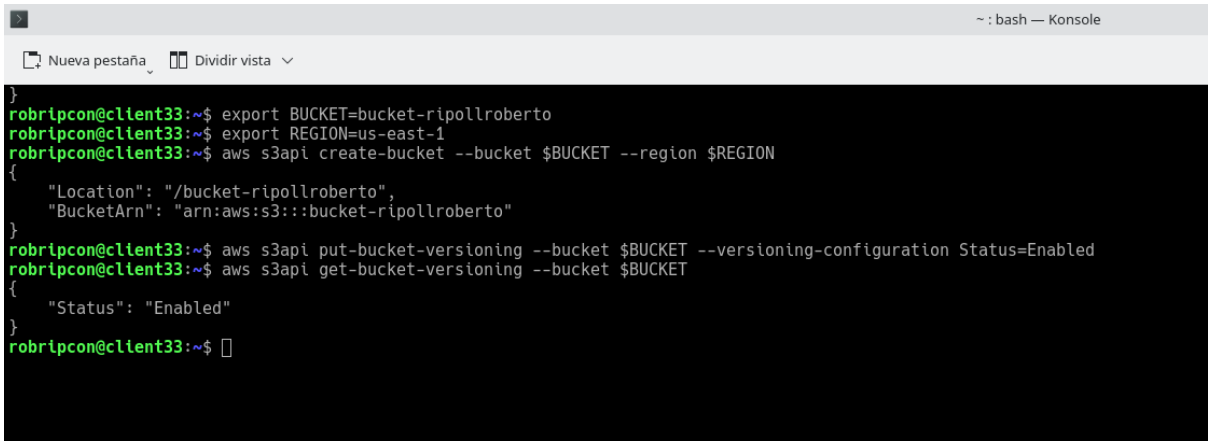
Objetivos: habilitar el versionado, recuperar versiones anteriores, entender los *delete markers*, restaurar un objeto "borrado" y limitar el coste con reglas de ciclo de vida.

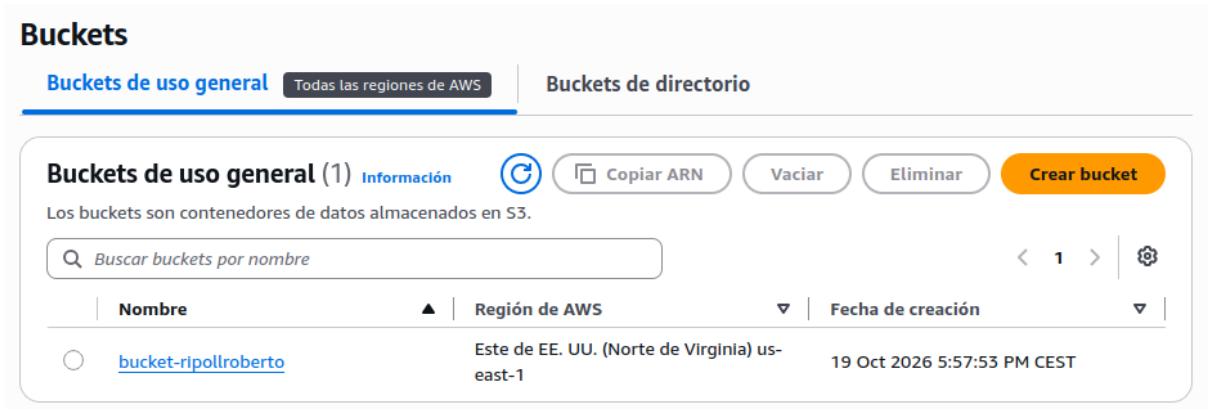
1. Crear el bucket y activar el versionado

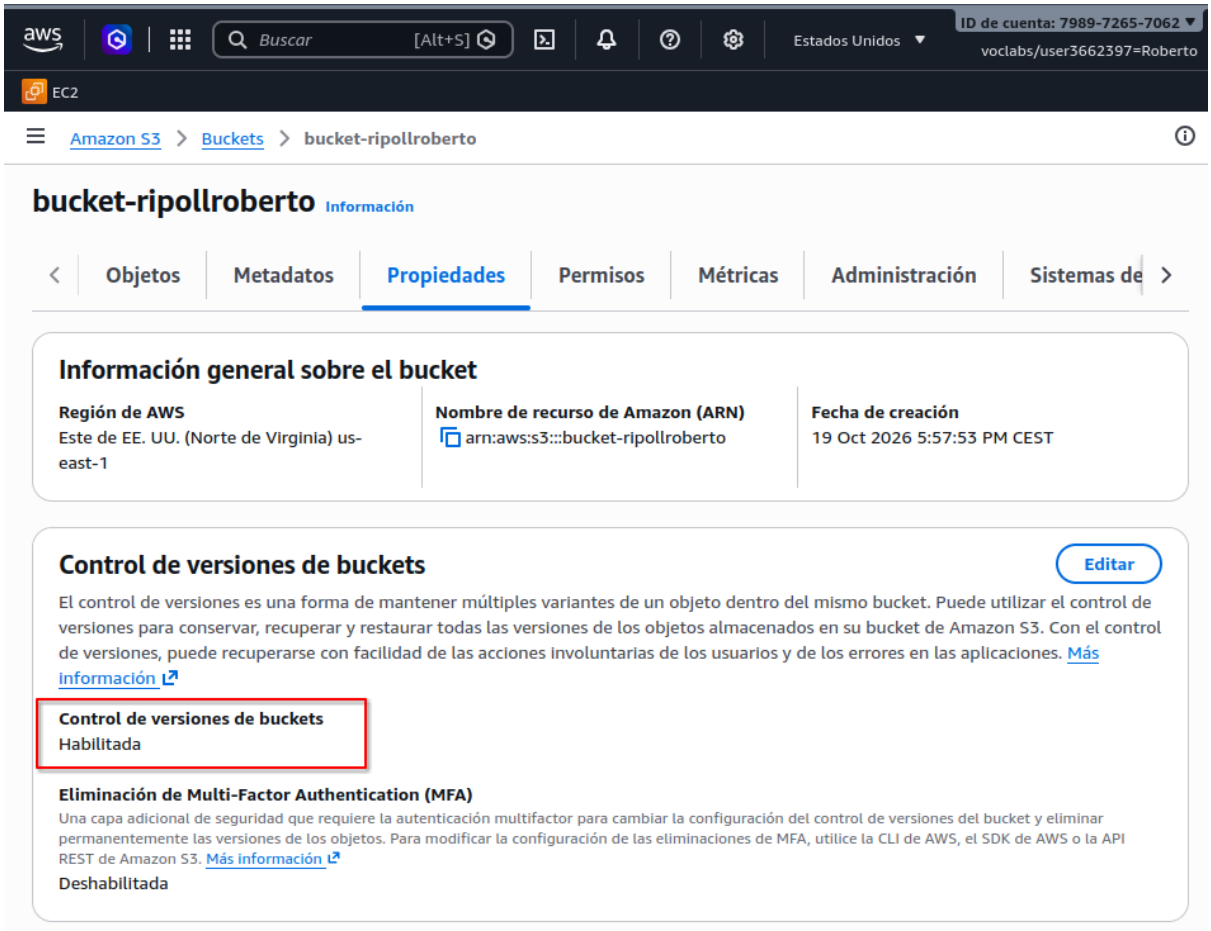
```
Export BUCKET=bucket-tunombre-numero
Export REGION=us-east-1
aws s3api create-bucket --bucket $BUCKET --region $REGION \ --region=$REGION
aws s3api put-bucket-versioning --bucket $BUCKET \ --versioning-configuration Status=Enabled
aws s3api get-bucket-versioning --bucket $BUCKET
```

✅ **Comprueba:** la salida debe mostrar "Status": "Enabled".

📸 Captura 1.






aws
Buscar
[Alt+S]
Estados Unidos
ID de cuenta: 7989-7265-7062
voclabs/user3662397=Roberto
EC2
Amazon S3 > Buckets > bucket-ripollroberto
bucket-ripollroberto Información
Objetos
Metadatos
Propiedades
Permisos
Métricas
Administración
Sistemas de
Información general sobre el bucket
Región de AWS
Este de EE. UU. (Norte de Virginia) us-east-1
Nombre de recurso de Amazon (ARN)
arn:aws:s3:::bucket-ripollroberto
Fecha de creación
19 Oct 2026 5:57:53 PM CEST
Control de versiones de buckets
Editar
El control de versiones es una forma de mantener múltiples variantes de un objeto dentro del mismo bucket. Puede utilizar el control de versiones para conservar, recuperar y restaurar todas las versiones de los objetos almacenados en su bucket de Amazon S3. Con el control de versiones, puede recuperarse con facilidad de las acciones involuntarias de los usuarios y de los errores en las aplicaciones. Más información
Control de versiones de buckets
Habilitada
Eliminación de Multi-Factor Authentication (MFA)
Una capa adicional de seguridad que requiere la autenticación multifactor para cambiar la configuración del control de versiones del bucket y eliminar permanentemente las versiones de los objetos. Para modificar la configuración de las eliminaciones de MFA, utilice la CLI de AWS, el SDK de AWS o la API REST de Amazon S3. Más información
Deshabilitada

2. Generar varias versiones de un objeto

```
echo "Informe v1" > informe.txt
aws s3 cp informe.txt s3://$BUCKET/informe.txt

echo "Informe v2 - cifras corregidas" > informe.txt
aws s3 cp informe.txt s3://$BUCKET/informe.txt

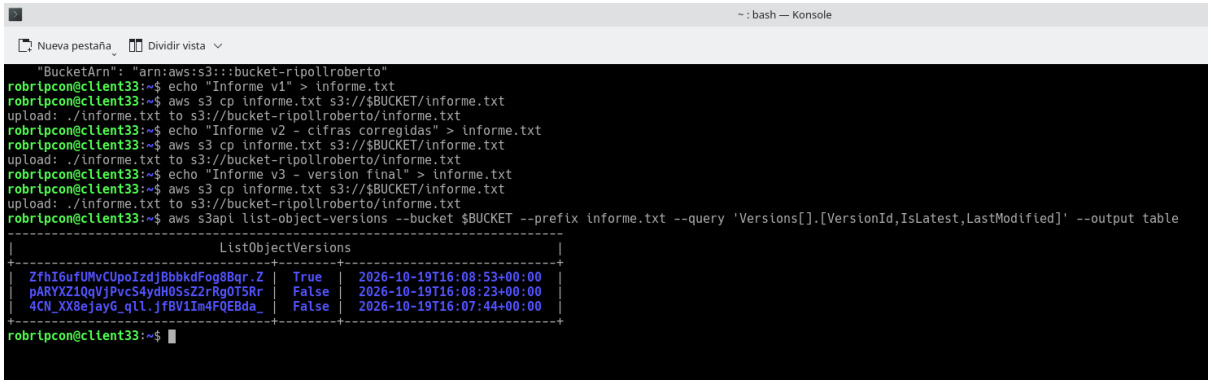
echo "Informe v3 - versión final" > informe.txt
aws s3 cp informe.txt s3://$BUCKET/informe.txt
```

Lista las versiones:

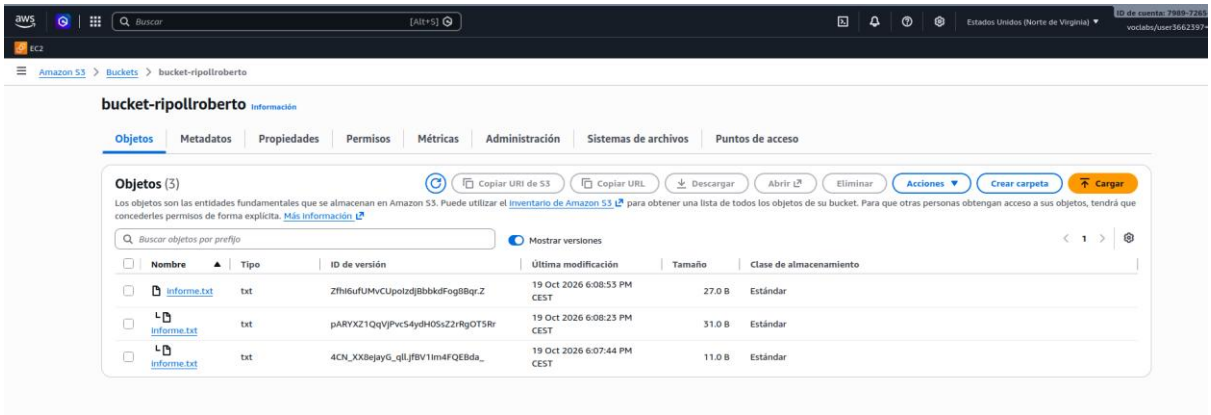
```
aws s3api list-object-versions --bucket $BUCKET --prefix informe.txt \
--query 'Versions[].[VersionId,IsLatest,LastModified]' --output table
```

📸 Captura 2.

✅ **Comprueba:** deben aparecer 3 versiones y solo una con IsLatest = True. **Anota el VersionId de la más antigua.**



ID de la más antigua: **4CN_XX8ejayG_qll.jfBV1Im4FQEBda_**



3. Recuperar una versión anterior

Descarga la v1 sin tocar la actual:

```
aws s3api get-object --bucket $BUCKET --key informe.txt --version-id \
<ID_VERSION_1> recuperado_v1.txt

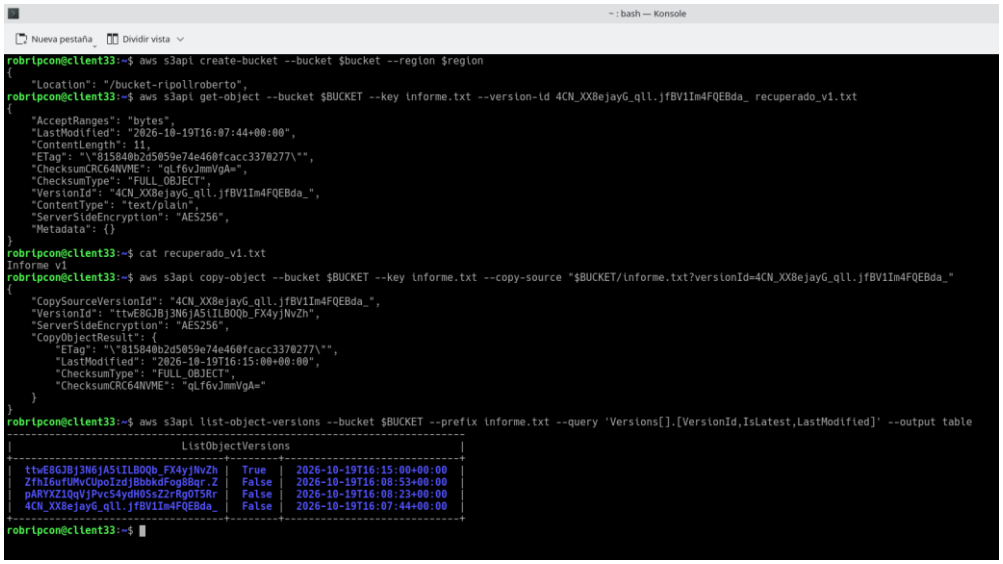
cat recuperado_v1.txt
```

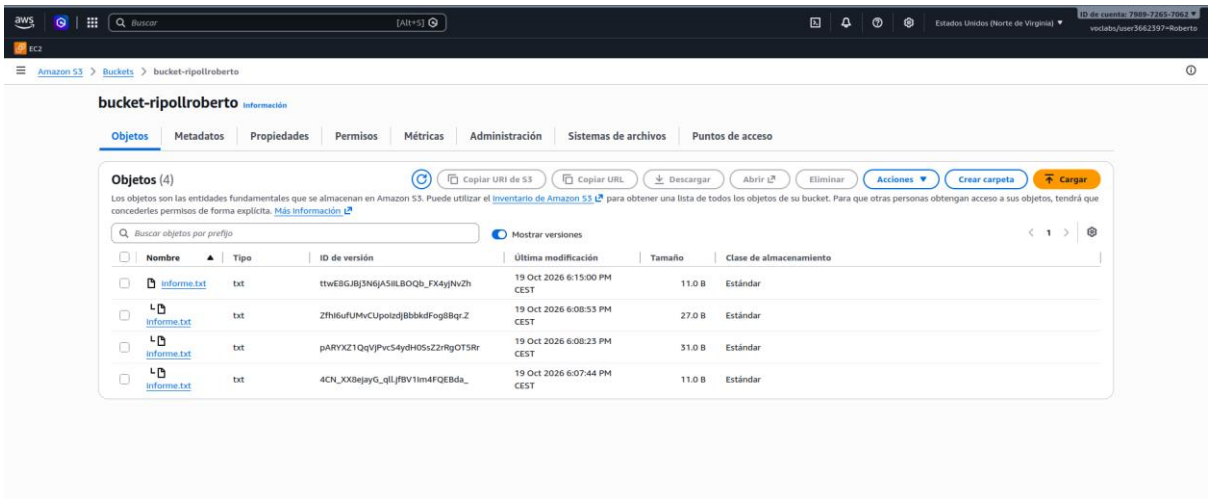
Ahora “restáurala” como versión actual copiándola sobre sí misma:

```
aws s3api copy-object --bucket $BUCKET --key informe.txt --copy-source \
"$BUCKET/informe.txt?versionId=<ID_VERSION_1>"
```

✅ **Comprueba:** vuelve a listar versiones. Ahora hay **4**, y la más reciente tiene el contenido de la v1. El versionado nunca sobrescribe: siempre añade.

📸 Captura 3.





4. Borrado y *delete markers*

```
aws s3 rm s3://$BUCKET/informe.txt
aws s3 ls s3://$BUCKET/
aws s3api get-object --bucket $BUCKET --key informe.txt salida.txt
```

El ls no muestra nada y el get-object da error NoSuchKey. Pero los datos siguen ahí:

```
aws s3api list-object-versions --bucket $BUCKET --query '{Versiones: Versions[].VersionId, Marcadores: DeleteMarkers[].[VersionId,IsLatest]}'
```

✅ **Comprueba:** verás las 4 versiones y un DeleteMarker con IsLatest = true.

📸 Captura 4.



5. Restaurar un objeto “borrado”

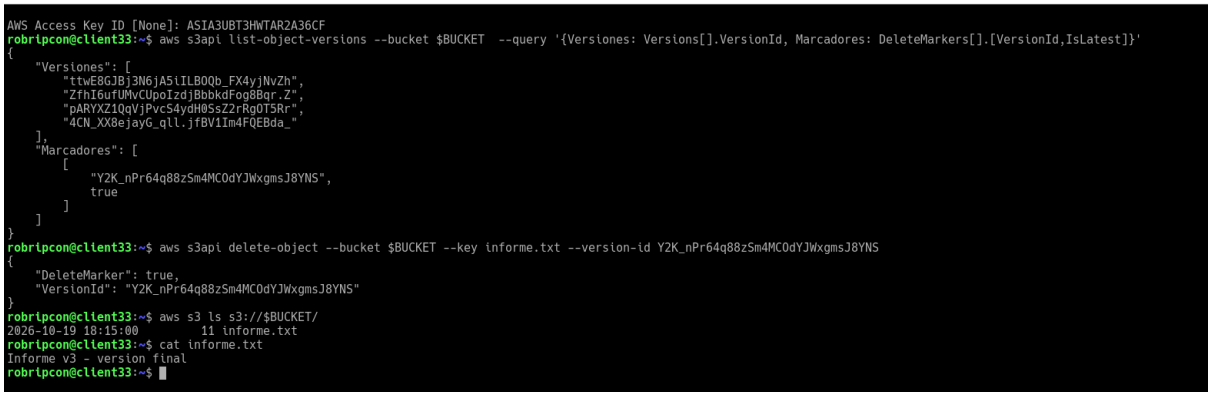
Para recuperarlo basta con eliminar el *delete marker*:

```
aws s3api delete-object --bucket $BUCKET --key informe.txt \
 --version-id <ID_DELETE_MARKER>
```

```
aws s3 ls s3://$BUCKET/
```

✅ **Comprueba:** informe.txt reaparece con su última versión.

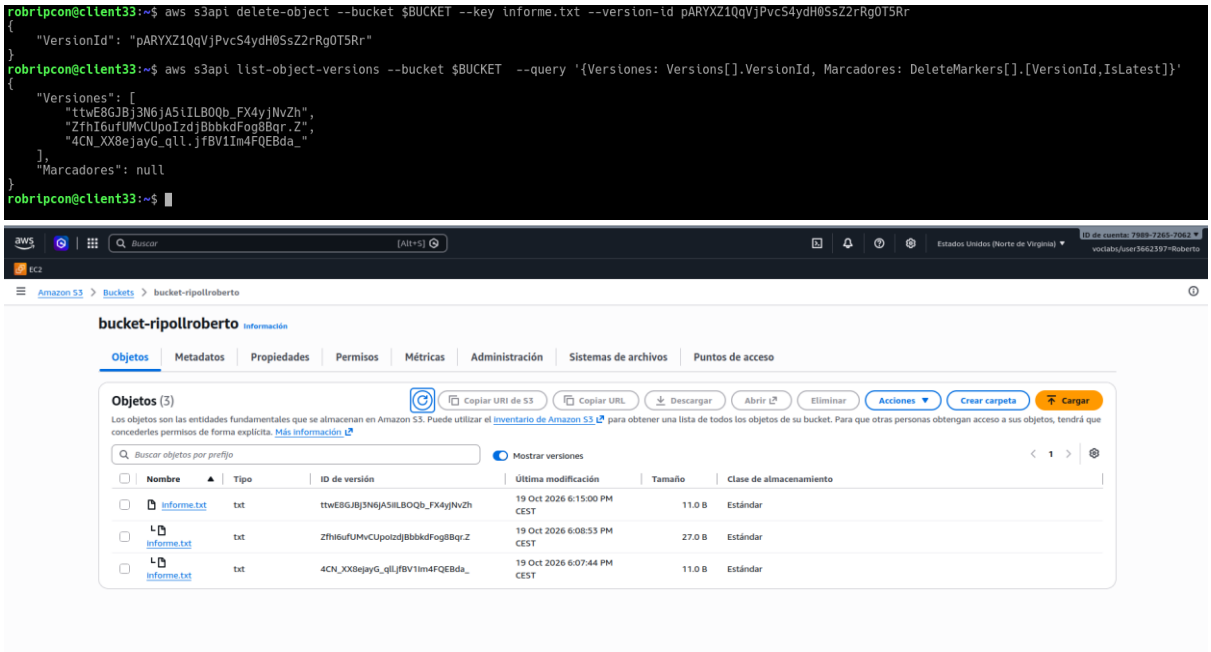
📸 Captura 5.



6. Borrado permanente de una versión concreta

```
aws s3api delete-object --bucket $BUCKET --key informe.txt \
 --version-id <ID_VERSION_2>
```

Al indicar --version-id, el borrado es **definitivo e irreversible** para esa versión. Compruébalo listando de nuevo.



7. Controlar el coste con una regla de ciclo de vida

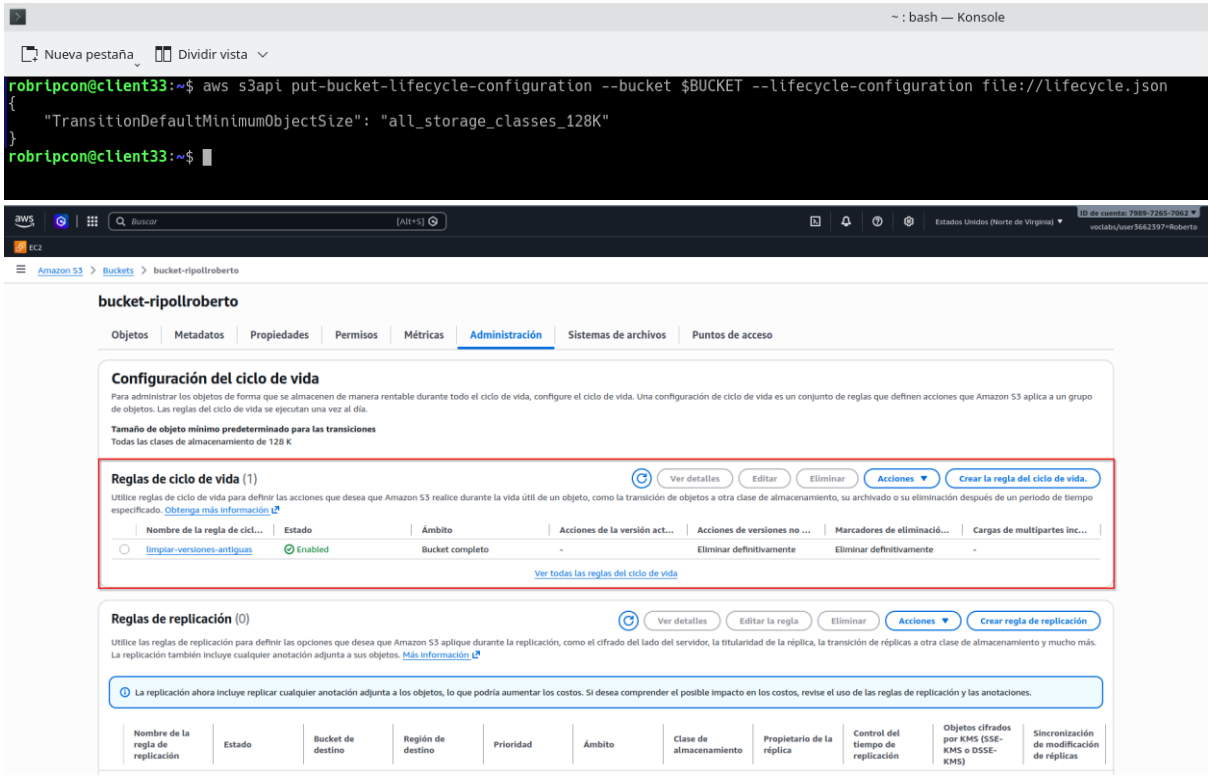
Las versiones antiguas se siguen facturando. Lo ideal es crear un ciclo de vida para controlar el coste. Crea el fichero lifecycle.json:

json

```
{
 "Rules": [
  {
   "ID": "limpiar-versiones-antiguas",
   "Status": "Enabled",
   "Filter": {},
   "NoncurrentVersionExpiration": {
    "NoncurrentDays": 30,
    "NewerNoncurrentVersions": 2
   },
   "Expiration": { "ExpiredObjectDeleteMarker": true }
  }
 ]
}
```

```
aws s3api put-bucket-lifecycle-configuration --bucket $BUCKET \
 --lifecycle-configuration file://lifecycle.json
```

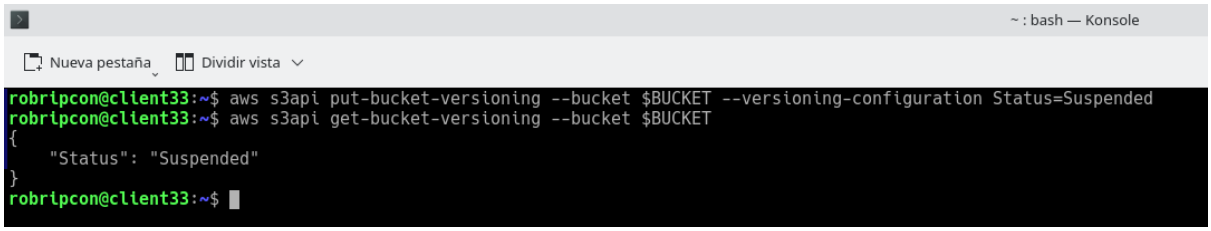
Esta regla conserva las 2 versiones no actuales más recientes y elimina las demás pasados 30 días. También limpia los *delete markers* huérfanos.



8. Suspender el versionado (opcional).

```
aws s3api put-bucket-versioning --bucket $BUCKET \
 --versioning-configuration Status=Suspended
```

Las versiones existentes **no se borran**. Los nuevos objetos se guardan con VersionId = null. Una vez habilitado, el versionado ya no se puede desactivar del todo, solo suspender.



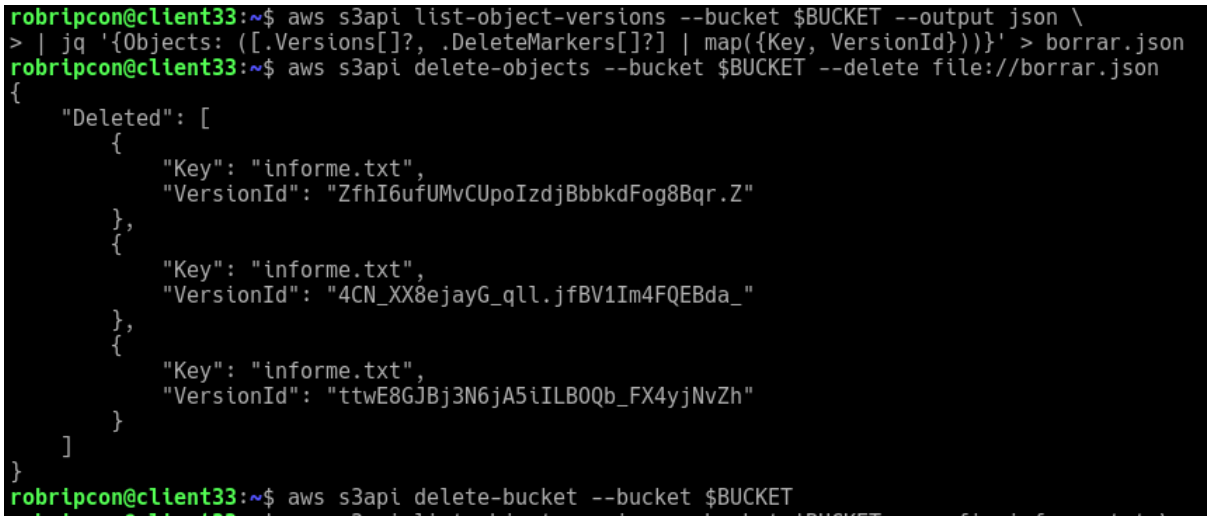
Limpieza (importante para evitar costes)

Un bucket con versionado no se puede borrar mientras tenga versiones o delete markers:

```
aws s3api list-object-versions --bucket $BUCKET --output json \
 | jq '{Objects: ([.Versions[]?, .DeleteMarkers[]?] | map({Key, VersionId}))}' >
```

borrar.json

aws s3api delete-objects --bucket $BUCKET --delete file://borrar.json
aws s3api delete-bucket --bucket $BUCKET



Preguntas:

1. ¿Por qué aws s3 rm no elimina realmente los datos en un bucket versionado?

Al estar habilitado el versionado cuando se elimina un objeto realmente se crea un marcador de eliminación.

2. ¿Qué diferencia hay entre borrar un objeto con y sin --version-id?

En caso de tener el bucket con versionado, se crea un marcador de eliminación como versión actual y el objeto deja de aparecer en las consultas normales, pero sus versiones anteriores siguen almacenadas. Por otro lado, si el bucket no tiene versionado se elimina el objeto permanentemente.

3. En la Parte 3, ¿por qué restaurar una versión crea una versión nueva en lugar de "volver atrás"?

En S3 el historial es inmutable entonces no permite retroceder, crea una copia exacta de la versión que se quiere recuperar.

4. ¿Qué pasaría con la factura si subes un archivo de 1 GB cien veces sin regla de ciclo de vida?

Aumentará el coste con cada nueva subida.

5. ¿Qué protección adicional ofrece **MFA Delete** y cuándo la usarías?

Añade versionado y múltiple factor de autenticación para realizar algunas operaciones, evita eliminaciones permanentes accidentales.